❺ To expose HTTP nodes' liveliness endpoints

❻ To find the nodes

This application starts an Akka cluster with a `Behavior.empty`. You don't need to add a function to this behavior to see how the cluster is formed; the business logic and the formation of a cluster are independent. In this example, `akka.remote.artery.canonical.hostname` and `akka.management.http.hostname` must be set to avoid conflicting IPs because the cluster is running locally. Bear in mind that this is for demo purposes only.

13.1.2 Cluster in action

To start the first node, enter the following command in the root of the Git repository:

```
$ sbt "chapter13a/runMain example.clustering2.App 1"
```

The result is the following output. It is a rich and understandable and is worth some attention:

```
No seed nodes found in configuration, relying on Cluster Bootstrap for
➥joining                                                                    ❶
Binding Akka Management (HTTP) endpoint to: 127.0.0.1:8558                   ❷
Bootstrap using `akka.discovery` method: config                              ❸
Locating service members. Using discovery [...], join decider
➥[....LowestAddressJoinDecider], scheme [http]                              ❹
Looking up [Lookup(testing-bootstrap,None,Some(tcp))]                        ❺
Located service members based on: [Lookup(testing-bootstrap,...],filtered
➥to [127.0.0.2:8558, 127.0.0.1:8558]                                        ❻

...

Bootstrap request from 127.0.0.1:50504: Contact Point returning 0 seed-      ❼
```